

Hummingbird-2 FPGA Implementation

Lightweight Authenticated Encryption for IoT Applications

Report

Author

Giorgos Ntakos

University of Patras

Computer Engineer & Informatics Department

Hardware Platform

Digilent Basys3 FPGA Development Board

Xilinx Artix-7 XC7A35T

Implementation Technologies

VHDL, Vivado, Python, UART Communication

June 17, 2026

Abstract

This report presents a complete FPGA implementation of the Hummingbird-2 lightweight authenticated encryption algorithm on a Digilent Basys3 development board featuring a Xilinx Artix-7 FPGA.

Unlike the original thesis implementation, the presented architecture was re-designed specifically for physical FPGA deployment and resource-efficient operation. The final system integrates a custom UART communication subsystem, clock domain crossing (CDC) mechanisms, a Python-based graphical user interface (GUI), automated validation utilities, benchmarking tools, and Monte Carlo reliability testing.

The implemented platform enables real-time encryption, decryption, message authentication code (MAC) generation, and MAC verification directly on physical FPGA hardware.

Experimental evaluation demonstrates successful timing closure at 250 MHz, low FPGA resource utilization, reliable operation across 10,000 Monte Carlo test iterations, and complete functional verification through dedicated validation vectors.

The resulting system provides a complete hardware/software co-design platform suitable for lightweight cryptographic research and FPGA-based IoT security applications.

Contents

1	Introduction	3
2	Project Evolution	3
2.1	FPGA-Oriented Architectural Redesign	4
3	System Architecture	5
4	Clocking Architecture	5
4.1	Clock Domain Crossing Implementation	6
5	UART Communication Protocol	6
5.1	Host-to-FPGA Packet	7
5.2	FPGA-to-Host Packet	7
6	Python Graphical User Interface	7
6.1	Cryptographic Interface	8
6.2	Session Logging	9
7	Validation Framework	10
7.1	Validation Results	10
8	Monte Carlo Reliability Testing	11
8.1	Test Configuration	12
8.2	Monte Carlo Results	12
9	FPGA Resource Utilization	13
10	Timing Analysis	14
11	Benchmark Results	14
11.1	Average Benchmark Results	15
11.2	Benchmark Visualization	16
12	HB2 Core Statistics	17
12.1	Core Performance Metrics	17
13	Conclusions	18
14	Future Work	19

1 Introduction

Lightweight cryptography has become increasingly important in Internet of Things (IoT) environments where devices operate under strict constraints in terms of computational resources, memory availability, power consumption, and implementation cost.

Traditional cryptographic algorithms often require computational resources that exceed the capabilities of many embedded and IoT platforms. As a result, lightweight cryptographic primitives have emerged as an important research area, aiming to provide adequate security while minimizing hardware and software overhead.

Hummingbird-2 is a lightweight authenticated encryption algorithm specifically designed for resource-constrained devices. The algorithm combines characteristics of both block ciphers and stream ciphers while simultaneously providing confidentiality and message authentication.

This project presents a complete FPGA-based implementation of the Hummingbird-2 algorithm using a Digilent Basys3 development board equipped with a Xilinx Artix-7 XC7A35T FPGA.

In addition to the cryptographic accelerator itself, the project includes:

- UART communication infrastructure
- Clock Domain Crossing (CDC) synchronization
- Python-based graphical user interface
- Automated validation framework
- Monte Carlo reliability testing
- Benchmarking infrastructure
- Real-time performance monitoring

The objective of this work is to demonstrate a complete end-to-end cryptographic platform capable of executing authenticated encryption operations directly on physical FPGA hardware while providing user-friendly software interaction and performance analysis capabilities.

2 Project Evolution

An earlier FPGA implementation of the Hummingbird-2 authenticated encryption algorithm was developed as part of previous thesis work.

While the original implementation successfully demonstrated the functionality of the algorithm on FPGA hardware, the architecture presented in this project introduces a significantly different design philosophy focused on practical deployment, reduced hardware utilization, and complete system integration.

The objective of the redesign was not only to implement the cryptographic algorithm itself but also to develop a complete hardware/software platform capable of operating on physical FPGA hardware while supporting communication, validation, benchmarking, and performance analysis.

The resulting implementation extends the original work by introducing:

- FPGA-oriented architectural redesign
- Resource sharing techniques
- UART communication support
- Clock Domain Crossing (CDC) synchronization
- Python GUI integration
- Automated validation framework
- Benchmarking infrastructure
- Monte Carlo reliability testing
- Real-time performance monitoring

2.1 FPGA-Oriented Architectural Redesign

The original implementation relied on a relatively high degree of hardware parallelism.

Although this approach reduced the number of required clock cycles, it increased FPGA resource consumption due to the large number of simultaneously instantiated processing blocks.

To improve resource efficiency, the architecture was redesigned using iterative processing and hardware reuse.

Table 1 summarizes the major architectural differences.

Table 1: Architectural Evolution

Original Architecture	FPGA Deployment Architecture
4 Parallel WD Blocks	1 Reusable WD Block
11 Parallel Encryption Blocks	1 Iterative Encryption Block
11 Parallel Decryption Blocks	1 Iterative Decryption Block
Parallel Processing	Round-Based Processing
Area-Oriented Design	Resource-Efficient Design

Instead of instantiating multiple hardware blocks simultaneously, the new architecture reuses a single hardware unit across multiple rounds of computation.

This design strategy significantly reduces FPGA resource utilization while maintaining the complete functionality of the Hummingbird-2 algorithm.

The trade-off introduced by this approach is an increase in execution latency due to the additional processing rounds required to complete each operation.

However, the resulting implementation is substantially more suitable for deployment on resource-constrained FPGA platforms.

3 System Architecture

The complete system consists of both hardware and software components working together as an integrated cryptographic platform.

Figure 1 illustrates the overall system dataflow.

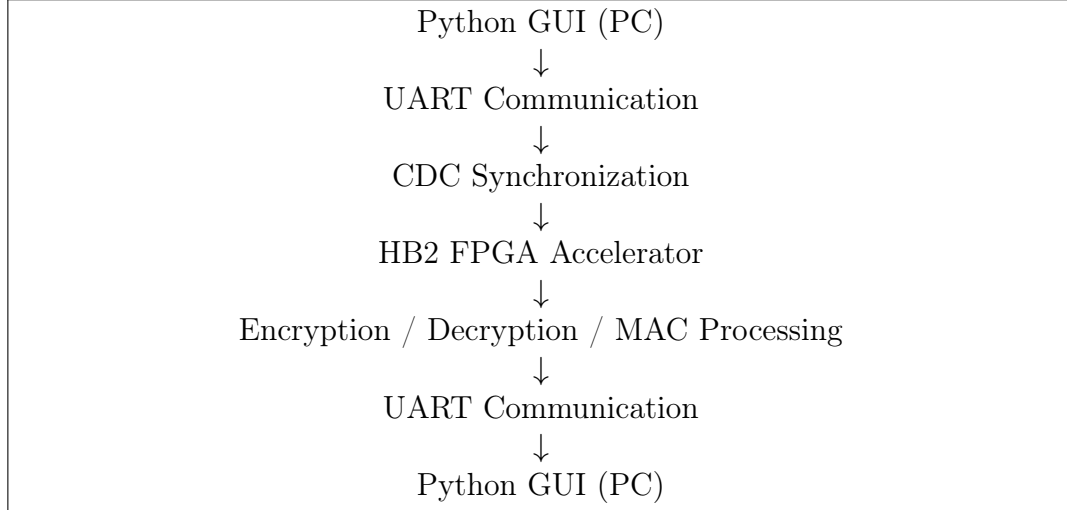


Figure 1: High-Level System Architecture

The FPGA design is composed of several major modules.

- UART Receiver
- UART Transmitter
- UART Packet Controller
- Initialization Module
- Full Encryption Engine
- Full Decryption Engine
- Clock Wizard
- CDC Synchronization Logic

The Python application provides a user-friendly interface that allows users to interact with the FPGA accelerator while also performing validation, benchmarking, and statistical analysis.

4 Clocking Architecture

The system utilizes multiple clock domains in order to separate communication logic from cryptographic processing.

The UART subsystem operates at a lower frequency while the cryptographic accelerator operates at a higher frequency to maximize processing performance.

Table 2 summarizes the clocking configuration.

Table 2: System Clock Domains

Clock Domain	Frequency
UART Logic	100 MHz
System Logic	100 MHz
HB2 Core	250 MHz

The 250 MHz clock is generated using the Vivado Clock Wizard IP core and is dedicated to the cryptographic accelerator.

This separation allows the computationally intensive Hummingbird-2 processing stages to execute at significantly higher frequencies than the communication subsystem.

4.1 Clock Domain Crossing Implementation

Because the UART subsystem and the Hummingbird-2 accelerator operate in different clock domains, dedicated synchronization logic is required to ensure reliable communication.

The implemented Clock Domain Crossing (CDC) architecture utilizes:

- Handshake-based synchronization
- Two-stage synchronizers
- Pulse-transfer logic
- Metastability mitigation techniques

The UART controller operates at 100 MHz while the HB2 core executes at 250 MHz.

Control requests generated by the UART subsystem are synchronized before entering the HB2 clock domain. Similarly, completion signals generated by the cryptographic accelerator are synchronized before being returned to the UART domain.

This approach guarantees reliable communication between asynchronous clock domains while preventing metastability-related failures.

The CDC implementation was extensively validated during hardware testing and forms a critical component of the final system architecture.

5 UART Communication Protocol

Communication between the host computer and the FPGA accelerator is performed through a custom UART-based communication protocol operating at a baud rate of 115200 bps.

The protocol was specifically designed to support authenticated encryption operations while providing a simple and reliable communication mechanism between the Python GUI and the FPGA hardware.

The communication flow consists of two packet types:

- Host-to-FPGA packet
- FPGA-to-Host packet

5.1 Host-to-FPGA Packet

The host transmits a 58-byte packet containing all information required by the cryptographic accelerator.

Table 3: Host-to-FPGA Packet Format

Field	Size
Flags	1 Byte
Text Length	1 Byte
Key	16 Bytes
Initialization Vector	8 Bytes
Data	16 Bytes
MAC	16 Bytes
Total	58 Bytes

The packet contains all information required for encryption, decryption, MAC generation, and MAC verification operations.

5.2 FPGA-to-Host Packet

After processing is completed, the FPGA transmits a response packet containing the generated results.

Table 4: FPGA-to-Host Packet Format

Field	Size
Output Data	16 Bytes
Generated MAC	16 Bytes
MAC Valid Flag	1 Byte
Latency Cycles	2 Bytes
Total	35 Bytes

The latency field allows direct measurement of cryptographic core performance and is used by the benchmarking infrastructure.

6 Python Graphical User Interface

A custom desktop application was developed using Python to simplify interaction with the FPGA accelerator.

The GUI provides a complete software interface for communication, validation, testing, and performance analysis.

Major features include:

- Encryption support
- Decryption support
- MAC generation

- MAC verification
- Session logging
- Validation execution
- Monte Carlo testing
- Benchmark execution
- Real-time statistics

6.1 Cryptographic Interface

The main application window allows users to configure keys, initialization vectors, plain-text data, ciphertext data, and authentication settings.

FPGA Connected

Operation

☒ Encrypt
 ☐ Decrypt

Key (32 hex chars)

23016745AB89EFCDDCFE98BA54761032

IV (16 hex chars)

34127856BC9AF0DE

Data

11003322554477669988BBAAADCCFFEE

MAC Tag

Text Length

128

☐ Integrity

Use Last Encryption Result

Test Vector

TV1

RUN

Data Output

D15BADF81423F420B1BAC2542945383D

MAC Tag Output

F6C4C0744BF6E721243776DC6CA61939

MAC STATUS: GENERATED

Last Encryption Result

Ciphertext:

D15BADF81423F420B1BAC2542945383D

MAC Tag:

F6C4C0744BF6E721243776DC6CA61939

Packet Viewer

===== TX PACKET =====

Flags : 00

Text Len : 128

Key:

23016745AB89EFCDDCFE98BA54761032

IV:

34127856BC9AF0DE

Data:

11003322554477669988BBAAADCCFFEE

MAC:

Figure 2: Main Cryptographic Interface

The interface provides direct access to encryption and decryption operations while displaying generated ciphertext, plaintext, and MAC values.

6.2 Session Logging

A dedicated logging subsystem records all important system events.

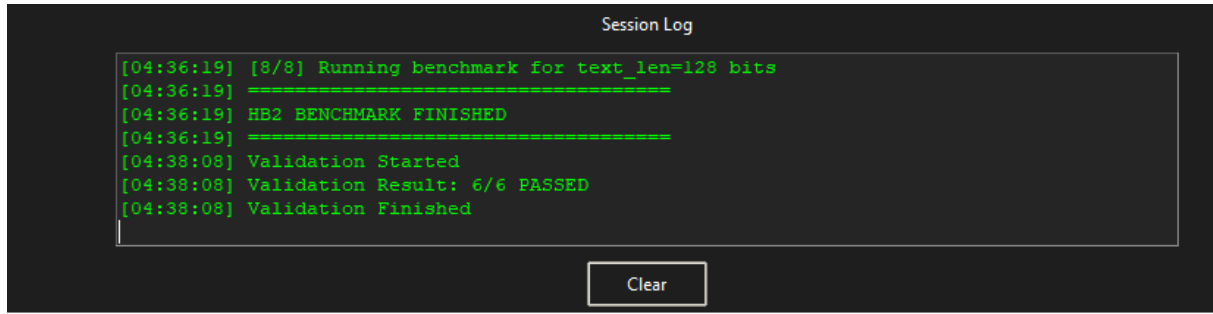


Figure 3: Session Logging Interface

The logging framework records:

- Encryption operations
- Decryption operations
- Validation execution
- Benchmark execution
- Monte Carlo execution
- Communication events

This functionality greatly simplifies debugging and system monitoring.

7 Validation Framework

A dedicated validation framework was developed to verify the correctness of the FPGA implementation.

The validation suite executes predefined test vectors and compares the generated FPGA outputs against expected reference values.

The framework automatically verifies:

- Encryption correctness
- Decryption correctness
- MAC generation
- MAC verification

7.1 Validation Results

Six independent test vectors were implemented and executed.

Table 5: Validation Results

Test Vector	Result
TV1	PASS
TV2	PASS
TV3	PASS
TV4	PASS
TV5	PASS
TV6	PASS

Overall validation result:

6 / 6 PASSED

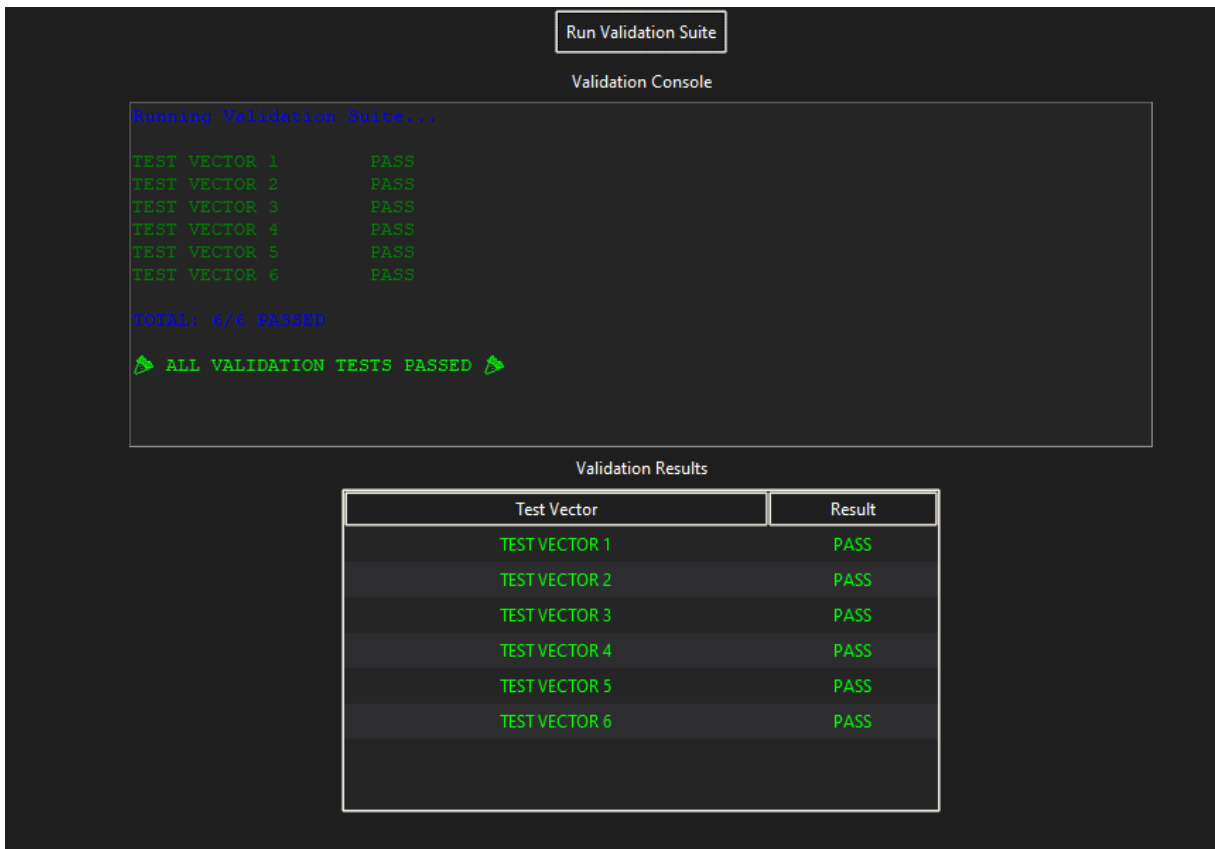


Figure 4: Validation Framework Results

The validation results confirm the functional correctness of the complete FPGA implementation.

8 Monte Carlo Reliability Testing

In addition to deterministic validation vectors, a Monte Carlo testing framework was implemented to evaluate long-term system reliability.

The framework performs repeated cryptographic operations using randomized inputs and automatically verifies output correctness.

8.1 Test Configuration

The Monte Carlo framework executes:

- Randomized encryption operations
- Randomized decryption operations
- Randomized keys
- Randomized initialization vectors
- Randomized plaintext inputs
- Automatic result verification

For the experiments presented in this report, a total of 10,000 iterations were executed.

8.2 Monte Carlo Results

Table 6: Monte Carlo Results

Metric	Value
Iterations	10000
Passed	10000
Failed	0
Success Rate	100%

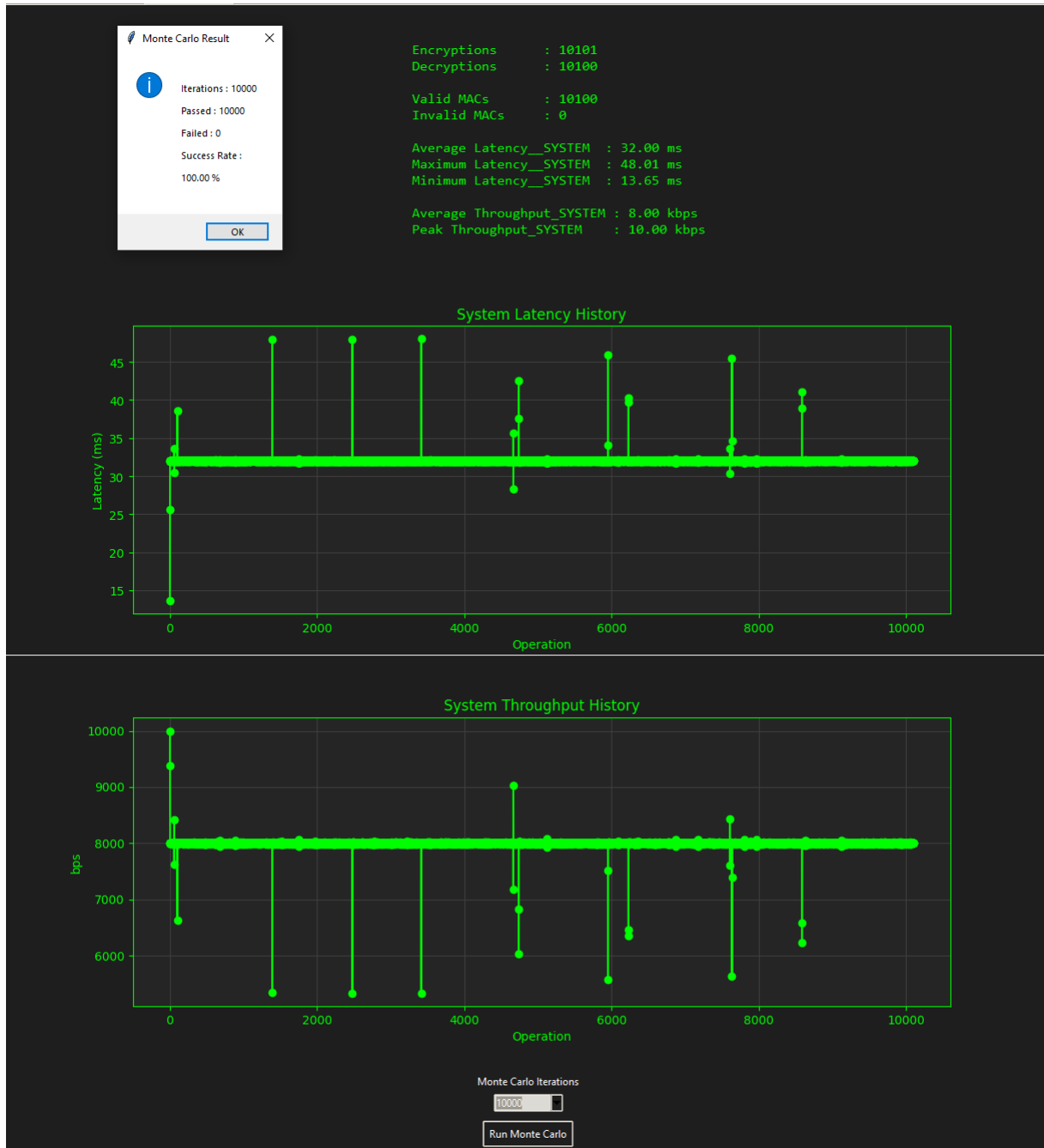


Figure 5: System Statistics Collected During Monte Carlo Execution

The successful completion of all 10,000 iterations demonstrates the stability and reliability of the implemented hardware/software platform.

9 FPGA Resource Utilization

Following synthesis and implementation in Xilinx Vivado, resource utilization statistics were collected for the target FPGA device.

The implementation was deployed on a Digilent Basys3 development board featuring a Xilinx Artix-7 XC7A35T FPGA.

Table 7 summarizes the final resource utilization.

Table 7: FPGA Resource Utilization

Resource	Used	Available	Utilization
LUTs	2533	20800	12.18%
Registers	4228	41600	10.16%
I/O	20	106	18.87%
MMCM	1	5	20.00%

The results demonstrate that the proposed architecture occupies only a small fraction of the available FPGA resources.

This relatively low utilization is a direct consequence of the iterative resource-sharing architecture adopted during the redesign process.

The remaining FPGA resources provide significant headroom for future extensions such as additional communication interfaces, embedded processors, or parallel cryptographic accelerators.

10 Timing Analysis

A complete timing analysis was performed following place-and-route implementation.

The system utilizes a multi-clock architecture consisting of:

- 100 MHz UART domain
- 100 MHz system domain
- 250 MHz HB2 cryptographic core domain

The 250 MHz clock is generated using the Vivado Clock Wizard IP core.

Table 8 summarizes the final timing results.

Table 8: Timing Summary

Metric	Value
HB2 Core Frequency	250 MHz
WNS	+0.001 ns
TNS	0.000 ns
Failing Endpoints	0

The timing analysis confirms successful timing closure across all clock domains.

No setup or hold violations were detected, demonstrating the effectiveness of both the architectural redesign and the CDC synchronization scheme.

11 Benchmark Results

A dedicated benchmarking framework was developed to evaluate the performance of the implemented cryptographic accelerator.

The benchmark executes multiple cryptographic operations using different message lengths and records execution latency, throughput, and processing cycles.

The evaluated message lengths were:

16, 32, 48, 64, 80, 96, 112, and 128 bits

11.1 Average Benchmark Results

Table 9 summarizes the average benchmark statistics.

Table 9: Average Benchmark Results

Metric	Value
Average Latency	0.0046 ms
Average Throughput	14515.94 kbps
Average Cycles	1147

The maximum measured throughput was:

19219.22 kbps

11.2 Benchmark Visualization

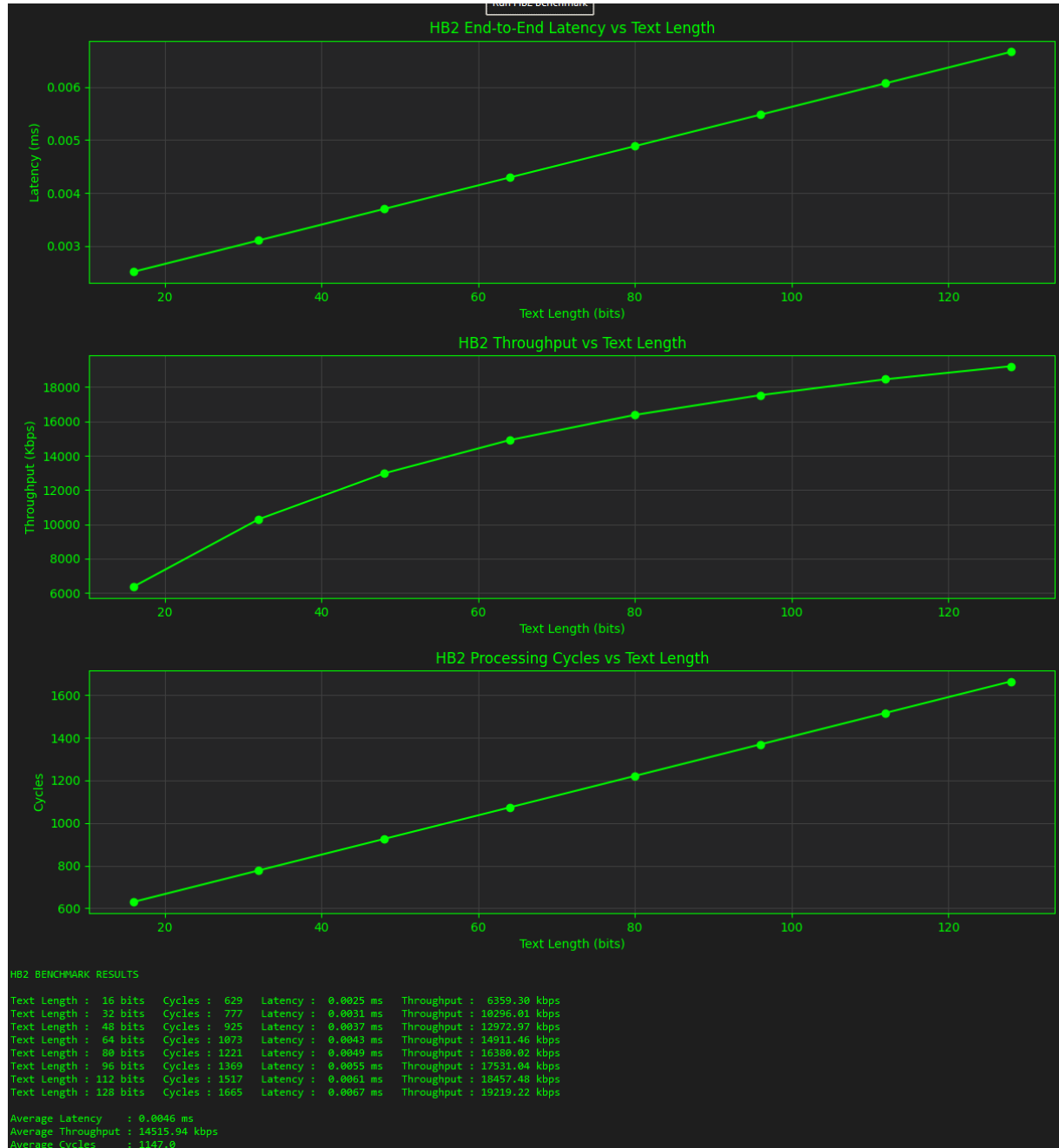


Figure 6: Benchmark Results for Different Message Lengths

The benchmark results indicate an approximately linear increase in processing latency as the message length grows.

This behavior is expected because the architecture reuses a single encryption and decryption datapath across multiple rounds of computation.

Consequently, larger messages require additional processing cycles.

Throughput increases for larger message sizes because the fixed initialization overhead becomes amortized across a greater amount of processed data.

These results demonstrate the trade-off between resource efficiency and performance introduced by the iterative architecture adopted for FPGA deployment.

12 HB2 Core Statistics

In addition to system-level benchmark measurements, dedicated statistics were collected directly from the cryptographic accelerator.

These measurements isolate the performance of the Hummingbird-2 core from UART communication overhead and software-side processing delays.

12.1 Core Performance Metrics

Table 10: HB2 Core Statistics

Metric	Value
Average Throughput	38438.40 kbps
Efficiency	44.54 kbps/Slice
Average Cycles	1664.9
Minimum Cycles	840
Maximum Cycles	1665

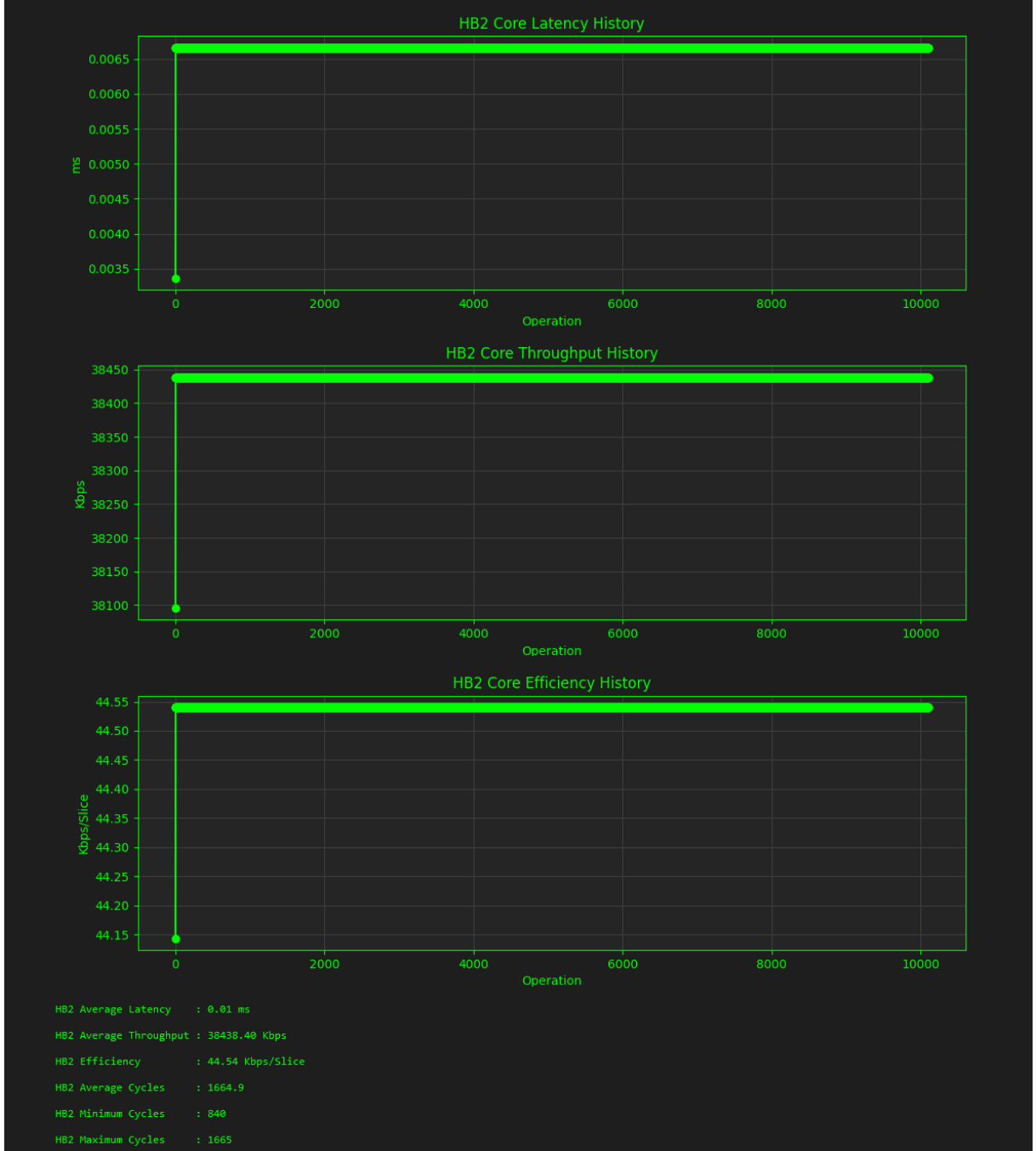


Figure 7: HB2 Core Performance Statistics

The collected statistics provide a more accurate representation of the actual cryptographic core performance because communication overhead is excluded from the measurements.

As a result, the reported throughput and cycle counts represent the true computational characteristics of the Hummingbird-2 accelerator.

13 Conclusions

This project successfully demonstrates a complete FPGA-based implementation of the Hummingbird-2 lightweight authenticated encryption algorithm on a physical Digilent Basys3 development board featuring a Xilinx Artix-7 FPGA.

The implementation extends the original thesis architecture through a redesigned resource-efficient datapath optimized for practical FPGA deployment.

Key contributions of this work include:

- FPGA-oriented architectural redesign
- Resource sharing and iterative processing
- UART communication infrastructure
- Clock Domain Crossing (CDC) support
- Desktop GUI integration
- Automated validation framework
- Benchmarking and performance analysis
- Monte Carlo reliability testing

The final implementation achieved successful timing closure at 250 MHz while utilizing only 12.18% of the available LUT resources of the target FPGA device.

Experimental evaluation confirmed correct functionality, successful MAC verification, stable operation across 10,000 Monte Carlo test iterations, and efficient utilization of FPGA resources.

The combination of hardware acceleration, multi-clock operation, software integration, automated testing, and performance monitoring resulted in a complete end-to-end lightweight cryptographic platform suitable for FPGA-based IoT security research and education.

14 Future Work

Potential future extensions of this work include:

- Integration of a MicroBlaze soft processor for embedded accelerator control
- Higher-bandwidth communication interfaces to overcome UART bandwidth limitations
- Throughput-oriented parallel HB2 architectures targeting high-performance applications
- Standalone embedded operation without requiring an external host computer
- Comparative FPGA evaluation against other lightweight cryptographic algorithms

These extensions would further enhance the flexibility, scalability, and applicability of the presented cryptographic platform.